

Session 4 · Deck 4.A4

Differentiation: Finite Differences to Autodiff

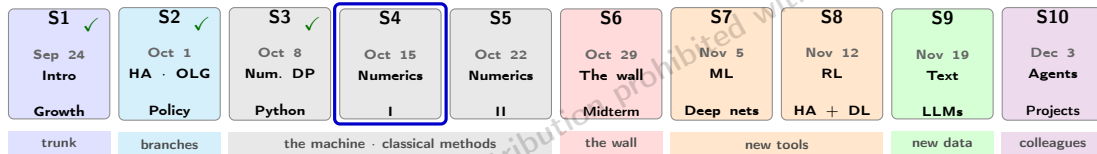
Zhigang Feng

Thursday, October 15, 2026

Build it on paper → solve it on the machine → push it to the frontier with AI

Where We Are

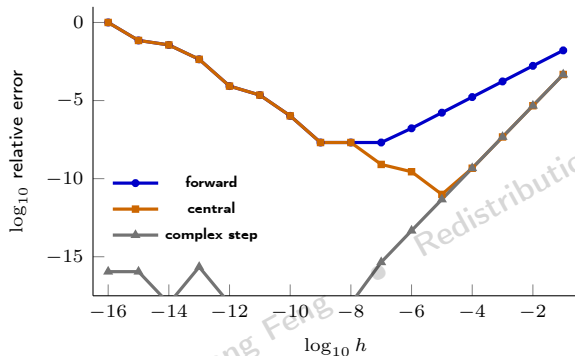
you are here: 4.A4 ▼



Build it on paper → solve it on the machine → push it to the frontier with AI

Block A: 4.A1 root-finding · 4.A2 optimization on an interval · 4.A3 optimization in many dimensions · 4.A4 differentiation.

Three Curves, One Derivative



The derivative of k^α at $k = 2$, computed three ways as the step shrinks.

	best log ₁₀ error
forward difference	-7.7
central difference	-11.0
complex step	exact

3.B3 showed you the first curve and called it a floor. This deck is where the floor gets an explanation — and where it turns out not to be a law of nature.

Why the Curve Has a Minimum

The forward difference carries two errors that move in opposite directions.

Truncation. Taylor gives

$$\frac{f(x+h) - f(x)}{h} = f'(x) + \frac{h}{2}f''(x) + O(h^2),$$

an error of order h — it *falls* with h .

Round-off. The numerator is a difference of two nearly equal numbers, each known to a relative $\varepsilon_{\text{mach}}$, so the quotient carries about $\varepsilon_{\text{mach}}|f|/h$ — it *rises* as h falls.

Balance them

Minimising $\frac{h}{2}|f''| + \varepsilon|f|/h$ gives

$$h^* \sim \sqrt{\varepsilon_{\text{mach}}} \approx 1.5 \times 10^{-8},$$

and an error of order $\sqrt{\varepsilon_{\text{mach}}}$ — **half your digits, gone.**

Measured: the best forward step was 10^{-7} and the best error $10^{-7.7}$. The theory predicts the location of the minimum, not just its existence.

Error decomposition adapted from QM *Numerical Methods*, "Numerical Differentiation: Error Analysis"; the cancellation mechanism is 3.A1's.

Central Differences: Better, and Twice the Price

Expand in both directions and the even-order terms cancel:

$$\frac{f(x+h) - f(x-h)}{2h} = f'(x) + \frac{h^2}{6} f'''(x) + O(h^4).$$

Truncation is now $O(h^2)$, so balancing against the same round-off gives

$$h^* \sim \varepsilon_{\text{mach}}^{1/3} \approx 6 \times 10^{-6},$$

with an error of order $\varepsilon_{\text{mach}}^{2/3}$ — about 10^{-11} .

	best h	best error
forward	10^{-7}	$10^{-7.7}$
central	10^{-5}	$10^{-11.0}$

Three and a half extra decades for one extra function evaluation per derivative.

In a Jacobian that doubling matters: $2N$ evaluations instead of $N + 1$. It is usually still worth it.

The Complex Step: Subtract Nothing, Lose Nothing

For a real-analytic f , expand along the *imaginary* axis:

$$f(x + ih) = f(x) + ihf'(x) - \frac{h^2}{2}f''(x) + \dots$$

Take the imaginary part and divide by h :

$$f'(x) = \frac{\operatorname{Im} f(x + ih)}{h} + O(h^2).$$

There is no subtraction, so there is no cancellation — and h can be made absurdly small.

Squire & Trapp (1998). Absent from every source deck in this course's corpus.

```
h = 1e-200
d = ((2.0 + 1j*h)**0.36).imag / h
# relative error: 0.0
```

At $h = 10^{-200}$ the answer is correct to **every bit**.
The price: f must accept a complex argument, so every abs, max and comparison inside it has to be complex-safe.

Automatic Differentiation: Neither Symbolic Nor Numerical

AD applies the chain rule to the program, one elementary operation at a time. No step size, no truncation, no cancellation — the result is exact to machine precision.

- **Forward mode** propagates a derivative alongside each value. Cost $\propto$ number of *inputs*.
- **Reverse mode** records the operations, then walks the tape backwards. Cost $\propto$ number of *outputs*.

A gradient has one output and many inputs, so reverse mode gives the whole thing for about the cost of **one** function evaluation. That is why it is what trains neural networks.

Which mode for a Jacobian?

N inputs and N outputs: forward mode costs N sweeps, reverse mode costs N sweeps. Neither wins, and structure — sparsity — is what actually helps.

AD differentiates **the program you wrote**, not the function you meant. Branches, clipping and `max` all have derivatives, and at a kink the one you get is whichever branch executed.

Mentioned once in passing in QM Lec. 4 as “may not be available”; the mechanics are 3.B3’s autograd tape.

Pay-Off: One Jacobian, Four Ways

The system from 4.A1, $x^2 + y^2 = 4$ and $e^x + y = 1$, whose Jacobian we can also write by hand.

	max error	evals
finite difference	3.7×10^{-8}	3
complex step	0	2
autograd	0	—
by hand	0	—

The finite-difference Jacobian is accurate to **eight digits**. The other two are accurate to sixteen.

Does it matter?

For Newton, often not: an approximate Jacobian costs iterations, not correctness, and Broyden works with a deliberately wrong one.

It matters when the derivative *is* the answer — Euler residuals, elasticities, standard errors, the gradient of a loss.

And it matters for the same reason 4.A3 did: eight digits of Jacobian caps how far Newton can converge.

Choosing

Method	Accuracy	Use it when
Forward difference	$\sqrt{\epsilon}$, about 8 digits	f is a black box and cheap
Central difference	$\epsilon^{2/3}$, about 11 digits	the default for a black box
Complex step	exact	you control the source and it is analytic
Automatic differentiation	exact	you control the source; gradients of many inputs
Symbolic / by hand	exact	the expression is small, and you will check it

The step size is not a free parameter to be tuned. It has a right answer, $\sqrt{\epsilon_{\text{mach}}}$ or $\epsilon_{\text{mach}}^{1/3}$, and choosing it by trial and error is how people end up on the wrong side of the U-curve.

Takeaways

- The finite-difference U-curve is **truncation falling plus round-off rising**. Its minimum sits at $h \sim \sqrt{\epsilon_{\text{mach}}}$ for forward and $\epsilon_{\text{mach}}^{1/3}$ for central differences — and the measurement lands where the theory says.
- A forward difference costs you **half your digits**, permanently. Central differences recover three and a half decades for one extra evaluation.
- The complex step removes the subtraction, and with it the floor: correct to every bit at $h = 10^{-200}$. The price is that f must be complex-safe.
- **Automatic differentiation is exact**, and in reverse mode a full gradient costs about one function evaluation — which is why it, not finite differences, is what makes Session 7 possible.
- AD differentiates the **program**, not the function. At a kink you get whichever branch ran.

Bridge: After the Break, Representing Functions

Block A has been about **operating** on a function you already have: finding where it is zero, where it is largest, how steep it is. Every one of those methods asked to evaluate f at a point of its choosing — and in a dynamic program, $\hat{v}$ is only known at N grid points.

So the questions change. How do you represent a function you only know at finitely many places? Which representation keeps the properties the algorithms of Block A depend on? And where does the transition matrix Π — which we have simply written down twice now — actually come from?

After the break (Block B)

4.B1 Approximation and interpolation.

4.B2 Quadrature: turning $\mathbb{E}$ into a finite sum.

4.B3 Discretizing an AR(1): Tauchen and Rouwenhorst, and which one you should actually use.

4.B4 The pay-off, measured.